

4x1 Multiplexer Module Code

```
module mux4x1 (sel, a, b, c, d, o);  
  
input [1:0] sel;  
  
input a, b, c, d;  
  
output reg o;  
  
always @(sel or a or b or c or d)  
  
begin  
  
    case (sel)  
  
        2'b00: o <= a;  
  
        2'b01: o <= b;  
  
        2'b10: o <= c;  
  
        2'b11: o <= d;  
  
    endcase  
  
end  
  
endmodule
```

4x1 Multiplexer Testbench Code

```
module testbench1();  
  
reg a, b, c, d;  
  
    reg [1:0] sel;  
  
reg y;  
  
// instantiate device under test  
  
    mux4x1 xyz(.sel(sel),.a(a), .b(b), .c(c),.d(d), .o(o) );  
  
// apply inputs one at a time  
  
initial begin // sequential block  
  
    $dumpfile("dump.vcd");
```

```
$dumpvars(1);

y=1'bx;

a = 1'b1; b = 1'b0; c = 1'bx; d=1'b1; #10;

sel = 2'b00; #10;

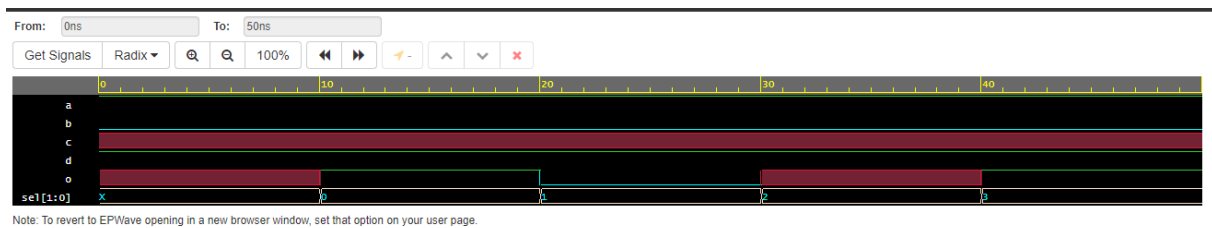
sel = 2'b01; #10;

sel = 2'b10; #10;

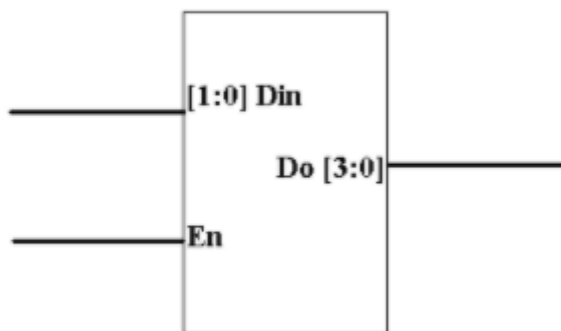
sel= 2'b11; #10;

end

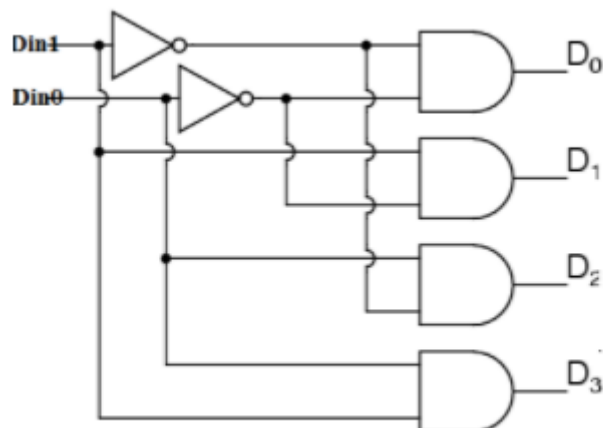
endmodule
```



2x4 Decoder Verilog Code



En	INPUTS		OUTPUTS			
	Din1	Din0	Do3	Do2	Do1	Do0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0



```

module decoder(Do, Din, En);
    input [1:0] Din;
    input En;
    output [3:0] Do;

    reg [3:0] Do;

    always @(En or Din)
    begin
        if (En)
            begin
                if (Din == 2'b00)
                    Do = 4'b0001;
                else if (Din == 2'b01)

```

```

        Do = 4'b0010;
    else if (Din == 2'b10)
        Do = 4'b0100;
    else if (Din == 2'b11)
        Do = 4'b1000;
    else
        $display("Error!");
    end
end
endmodule

```

Decoder Testbench

```

module decoder_tb_v;

    reg [1:0] Din;

    reg En;

    wire [3:0] Do;

    decoder uut(
        .Do(Do),
        .Din(Din),
        .En(En)
    );

    initial begin

        $dumpfile("dumpx.vcd");

        $dumpvars(1);

        // Initialize Inputs

        En = 1;

        Din =2'b00; #20;
    end
endmodule

```

```
Din = 2'b01; #20;
```

```
Din = 2'b10; #20;
```

```
Din = 2'b11; #20;
```

```
end
```

```
endmodule
```